

Chapter 4: Your Deep Learning Development Environment - Python, Google Colab, and Essential Tools

Printing and a simple conditional

In [1]:

```
print("Hello, Python!")
if 5 > 3:
    print("Five is greater than three!")
```

Hello, Python!
Five is greater than three!

Variables and basic types

In [2]:

```
x = 42
print(type(x))  # Output: <class 'int'>
```

<class 'int'>

Arithmetic and type conversion

In [3]:

```
apples = 2 + 3
print(apples)  # Output: 5
num = "10"     # This is a string
num = int(num) # Now it's an integer
print(num * 2) # Output: 20
```

5
20

Standard library math (square root)

In [4]:

```
import math
number = 16
root = math.sqrt(number)
print("The square root of", number, "is", root)
```

The square root of 16 is 4.0

NumPy basics (array and mean)

In [5]:

```
import numpy as np  # We'll explain more about NumPy later

arr = np.array([10, 20, 30, 40])
print("Mean of array:", np.mean(arr))  # Calculate the mean (average)
```

Mean of array: 25.0

Control flow (if/else and while loop)

In [7]:

```
age = 16
if age >= 18:
    print("You can vote!")
else:
    print("You cannot vote yet.")

energy = 5
while energy > 0:
    print("Keep running!")
    energy -= 1
```

You cannot vote yet.
Keep running!
Keep running!
Keep running!
Keep running!
Keep running!

Functions with parameters and f-strings

In [8]:

```
def make_sandwich(filling):
    # f-strings insert variable values directly: {filling}
    print(f"Making a {filling} sandwich!")

make_sandwich("tuna")
make_sandwich("grilled cheese")

def greet_person(name):
    print(f"Hello, {name}! Great to see you.")

greet_person("Alice")
greet_person("Bob")

# A small labeled print to show where we are
print("--- Example: Using f-strings ---")
make_sandwich("tuna")
make_sandwich("grilled cheese")
```

Making a tuna sandwich!
Making a grilled cheese sandwich!
Hello, Alice! Great to see you.
Hello, Bob! Great to see you.
--- Example: Using f-strings ---
Making a tuna sandwich!
Making a grilled cheese sandwich!

Lambdas and callables

In [9]:

```
# Lambda vs def
square = lambda x: x * x
print(square(4)) # Output: 16

def square_def(x):
    return x * x

print(square_def(4)) # Output: 16

add = lambda a, b: a + b
```

```

print(add(3, 7))    # Output: 10

def add_def(a, b):
    return a + b

print(add_def(3, 7))    # Output: 10

# What is "callable"?
def say_hello():
    print("Hello!")

print(callable(say_hello))    # True
say_hello()                   # Hello!

class Counter:
    def __call__(self):
        print("Counting...")

c = Counter()
print(callable(c))    # True, because Counter implements __call__
c()

# Lambdas are callable too
square = lambda x: x * x
print(callable(square))    # True
print(square(5))           # 25

```

```

16
16
10
10
True
Hello!
True
Counting...
True
25

```

PyTorch callables and custom activations

In [10]:

```

import torch
import torch.nn as nn

# nn.ReLU is a callable object (it has __call__)
relu = nn.ReLU()
input_tensor = torch.tensor([-1.0, 0.0, 2.0])
output_tensor = relu(input_tensor)    # Calls __call__ internally, applies ReLU
print(output_tensor)    # tensor([0., 0., 2.])

# Custom activation as a function
def custom_activation(x):
    return torch.maximum(x, torch.tensor(0.0))

out = custom_activation(torch.tensor([-2.0, 3.0]))
print(out)    # tensor([0., 3.])

# Custom activation as a callable class
class CustomReLU:
    def __call__(self, x):
        return torch.maximum(x, torch.tensor(0.0))

custom_relu = CustomReLU()
result = custom_relu(torch.tensor([-3.0, 4.0]))
print(result)    # tensor([0., 4.])

```

```

tensor([0., 0., 2.])
tensor([0., 3.])
tensor([0., 4.])

```

Variable arguments (*args)

In [12]:

```
def shopping_list(*items):  
    print("Items to buy:")  
    for item in items:  
        print(item)  
  
shopping_list("apples", "bread", "milk")
```

Items to buy:
apples
bread
milk

Returning values from functions

In [13]:

```
def add(a, b):  
    return a + b  
  
sum_value = add(3, 5)  
print(sum_value)  # 8  
  
def full_name(first, last):  
    return f"{first} {last}"  
  
name = full_name("Ada", "Lovelace")  
print(name)
```

8
Ada Lovelace

File I/O (write then read)

In [14]:

```
# Define some names  
names = ["Alice", "Bob", "Charlie"]  
  
# Write to a file  
with open("names.txt", "w") as file:  
    for name in names:  
        file.write(name + "\n")  
print("File written successfully.")  
  
# Read from the file  
with open("names.txt", "r") as file:  
    lines = file.readlines()  
  
# Print lines without trailing newline characters  
for line in lines:  
    print(line.strip())
```

File written successfully.
Alice
Bob
Charlie

NumPy arrays: vectorization, dtypes, structured arrays, shape, transpose, slicing

In [15]:

```
import numpy as np

# Vectorized math
numbers = np.array([1, 2, 3, 4])
print(numbers * 2)  # [2 4 6 8]

# Python list can hold mixed types
my_list = [10, "hello", 3.14, True, [1, 2], {"name": "Alice"}]
print(my_list)
for item in my_list:
    print(type(item))

# NumPy arrays have one dtype; mixed inputs get upcast
arr1 = np.array([1, "hello", 3.14])
print(arr1)
print(arr1.dtype)  # likely <U... (string)

arr2 = np.array([1, 2.5, True])
print(arr2)
print(arr2.dtype)  # float64 (True becomes 1.0)

# Structured arrays (like small tables)
person_dtype = np.dtype([
    ('name', 'U10'),
    ('age', 'i4'),
    ('height', 'f8')
])

people = np.array([
    ('Alice', 30, 1.65),
    ('Bob', 24, 1.80),
    ('Charlie', 35, 1.75)
], dtype=person_dtype)

print(people['name'])
print(people[0]['name'])

# Shape and transpose
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix.shape)  # (2, 3)
print(matrix.T)      # transpose
print(matrix.shape)  # shape is unchanged for the original

# Indexing and slicing
arr = np.array([10, 20, 30, 40, 50])
print(arr[0])        # 10
print(arr[-1])       # 50
print(arr[1:4])      # [20 30 40]
```

```
[2 4 6 8]
[10, 'hello', 3.14, True, [1, 2], {'name': 'Alice'}]
<class 'int'>
<class 'str'>
<class 'float'>
<class 'bool'>
<class 'list'>
<class 'dict'>
['1' 'hello' '3.14']
<U32
[1.  2.5 1. ]
float64
['Alice' 'Bob' 'Charlie']
Alice
(2, 3)
[[1 4]
 [2 5]
 [3 6]]
(2, 3)
10
50
```

```
[20 30 40]
```

Membership tests in containers

In [16]:

```
# Sets store unique items
fruits = {"apple", "banana", "cherry", "apple"}
print(fruits)  # duplicates removed

# Membership with sets
print("banana" in fruits)  # True

# Membership with other containers
fruits = {"apple", "banana", "cherry"}
print("banana" in fruits)  # True
print("grape" in fruits)   # False

my_list = [10, 20, 30, 40]
print(20 in my_list)      # True
print(50 in my_list)      # False

my_tuple = ("red", "green", "blue")
print("green" in my_tuple) # True
print("yellow" in my_tuple) # False

my_string = "hello world"
print("world" in my_string) # True
print("bye" in my_string)   # False
print("o w" in my_string)   # True (spaces count)

my_dict = {"name": "Alice", "age": 30}
print("name" in my_dict)    # True (checks keys)
print("gender" in my_dict)  # False
print("Alice" in my_dict)   # False (values are not checked by default)
```

```
{'apple', 'banana', 'cherry'}
True
True
False
True
False
True
False
True
False
True
False
True
True
False
False
```

Exceptions with try/except

In [17]:

```
# Example 1: Handling ZeroDivisionError
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Oops! You can't divide by zero.")

# Example 2: Handling FileNotFoundError
try:
    with open("data.txt", "r") as file:
        content = file.read()
        print(content)
except FileNotFoundError:
```

```
print("File not found. Please check the filename.")
```

Oops! You can't divide by zero.
File not found. Please check the filename.

OOP – building a Robot class step by step

In [18]:

```
# Basic Robot
class Robot:
    def __init__(self, name, purpose):
        self.name = name
        self.purpose = purpose

    def introduce(self):
        return f"Greetings! I am {self.name}, designed for {self.purpose}."

# Create objects and use a method
robohelper = Robot("RoboHelper", "cleaning")
medbot = Robot("MedBot", "healthcare")

print(robohelper.introduce())
print(medbot.introduce())

# Extend Robot with battery_life and a different 'function' label
class RobotWithBattery:
    def __init__(self, name, function, battery_life):
        self.name = name
        self.function = function
        self.battery_life = battery_life

    def introduce(self):
        return (
            f"Greetings! I am {self.name}, specialized in {self.function} "
            f"with {self.battery_life}% battery remaining."
        )

cleaner = RobotWithBattery("Cleaner3000", "floor cleaning", 85)
print(cleaner.introduce())

# Add behavior: recharge
class RechargeableRobot:
    def __init__(self, name, function, battery_life):
        self.name = name
        self.function = function
        self.battery_life = battery_life

    def introduce(self):
        return (
            f"Greetings! I am {self.name}, specialized in {self.function} "
            f"with {self.battery_life}% battery remaining."
        )

    def recharge(self):
        self.battery_life = 100
        return f"{self.name} is now fully recharged!"

r1 = RechargeableRobot("RoboHelper", "cleaning", 70)
r2 = RechargeableRobot("MedBot", "healthcare", 45)
r3 = RechargeableRobot("Cleaner3000", "floor cleaning", 25)

print(r1.introduce())
print(r1.recharge())
print(r2.introduce())
print(r2.recharge())
print(r3.introduce())
print(r3.recharge())
```

Greetings! I am RoboHelper, designed for cleaning.

Greetings! I am MedBot, designed for healthcare.

Greetings! I am MedBot, designed for healthcare.
Greetings! I am Cleaner3000, specialized in floor cleaning with 85% battery remaining.
Greetings! I am RoboHelper, specialized in cleaning with 70% battery remaining.
RoboHelper is now fully recharged!
Greetings! I am MedBot, specialized in healthcare with 45% battery remaining.
MedBot is now fully recharged!
Greetings! I am Cleaner3000, specialized in floor cleaning with 25% battery remaining.
Cleaner3000 is now fully recharged!

Encapsulation via "private" attribute (name-mangled)

In [19]:

```
class RobotSecure:
    def __init__(self, name, function, battery_level):
        self.name = name
        self.function = function
        self.__battery_level = battery_level  # private-like

    def recharge(self, amount):
        self.__battery_level = min(100, self.__battery_level + amount)
        return f"{self.name} recharged to {self.__battery_level}% battery."

    def work(self, hours):
        energy_spent = hours * 10
        if energy_spent > self.__battery_level:
            return f"{self.name} does not have enough battery to work for {hours} hours."
        self.__battery_level -= energy_spent
        return f"{self.name} worked for {hours} hours. Battery now at {self.__battery_level}%"

    def get_battery_status(self):
        return f"{self.name} has {self.__battery_level}% battery remaining."

sweeper = RobotSecure("SweeperBot", "cleaning", 50)
print(sweeper.get_battery_status())
print(sweeper.work(3))
print(sweeper.recharge(40))
print(sweeper.get_battery_status())
```

SweeperBot has 50% battery remaining.
SweeperBot worked for 3 hours. Battery now at 20%
SweeperBot recharged to 60% battery.
SweeperBot has 60% battery remaining.

PyTorch nn.Module – simple linear model

In [20]:

```
import torch
import torch.nn as nn

class SimpleLinearModel(nn.Module):
    def __init__(self):
        super().__init__()  # initialize base class
        self.linear = nn.Linear(3, 1)  # 3 inputs -> 1 output

    def forward(self, x):
        return self.linear(x)

model = SimpleLinearModel()
example_input = torch.tensor([[1.0, 2.0, 3.0]])
output = model(example_input)
print(output)

tensor([[ -0.5580]], grad_fn=<AddmmBackward0>)
```


Constants and naming conventions (snake_case, PascalCase)

In [21]:

```
PI = 3.14159    # constant (ALL_CAPS by convention)

def calculate_area(radius):    # snake_case for functions
    return PI * radius * radius

class Circle:    # PascalCase for classes
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return calculate_area(self.radius)

c = Circle(5)
print(c.area())
```

78.53975

Final function call example

In [22]:

```
def greet_and_start():
    print("Hello!")
    print("Program starting...")
```

```
greet_and_start()
```

```
Hello!
Program starting...
```

In []: